5 سارة محمد مدالله المجالي

الواجهة الرسومية والـ Gantt chart

📌 شو إلك بالمشروع

- **Phase 2 · فيديو 19 نوفمبر:** Gantt chart بسيط للـ CPU timeline
- **Phase 3 · 17 ديسمبر:** الواجهة الكاملة والـ Gantt المتحرك
- **Phase 4 · 31 ديسمبر:** الحفظ والتحميل والتصدير

الكود كله بملف **CPU_SchedLab.py**. افتحه بـ VS Code واكبس **Ctrl + G** واكتب رقم السطر اللي بالصورة.

Phase 2 · فيديو 19 نوفمبر — Gantt chart بسيط للـ CPU timeline

الـ Gantt chart البسيط (F8)

CPU_SchedLab.py — lines 375–382

```
375  def text_gantt(timeline):
376      """F8 (Phase 2) the SIMPLE Gantt chart as text:  | P1 | P2 |  then the times."""
377      bar, axis = "|", "0"
378      for s in timeline:
379          width = max(len(s.pid) + 2, (s.end - s.start) * 2)
380          bar += ("--" if s.pid == IDLE else s.pid).center(width) + "|"
381          axis += str(s.end).rjust(width + 1)
382      return bar + "\n" + axis
```

🎤 شو بتحكي للدكتورة

الـ **Gantt chart** بيرسم الخط الزمني تبع المعالج: كل مستطيل هو process، وطوله قد المدة اللي اشتغلتها. هاي النسخة الأولى منه: الدالة **text_gantt** بتمرق على أجزاء الخط الزمني، ولكل جزء بتكتب اسم الـ process بعرض على قد مدّتها، وتحتها بتكتب وقت ما خلصت. والـ idle بيتكتب كخط فاضي. ولما نكبس Run، بينطبع هالرسم بالـ Terminal.

هيك بيطلع بالـ Terminal

Terminal

```
FCFS
|    P1    |  P2  | P3 |
0          5      8    9

SRTF
| P1 | P2 | P3 | P2 |   P1   |
0    1    2    3    5        9
```

🎤 شو بتحكي للدكتورة

هاد الرسم للمثال تبعنا: بـ FCFS، P1 من 0 لـ 5، P2 لـ 8، P3 لـ 9. وبـ SRTF بنشوف المقاطعات. يعني من الرسم البسيط بتقدري تعرفي مين اشتغل وإمتى، وكل الأرقام بنحسبها منه.

💬 إذا سألتك الدكتورة

❓ شو الـ Gantt chart؟

رسمة بتبيّن أي process كانت على المعالج بكل لحظة.

❓ ليش عرض كل مستطيل مختلف؟

لأنه على قد المدة اللي اشتغلتها الـ process.

الـ Gantt المتحرك

CPU_SchedLab.py lines 402–415

```python
    def play(self, result, seconds):
        """Draw the chart little by little: 40 frames per second, `seconds` long."""
        if self.job:
            self.after_cancel(self.job)
        self.result, end = result, result.end_time
        self.now = 0.0 if seconds > 0 else end
        step = end / max(1, seconds * 40)

        def tick():
            self.now = min(end, self.now + step)
            self.redraw()
            if self.now < end:
                self.job = self.after(25, tick)              # next frame (window stays free)
        tick()
```

🎤 شو بتحكي للدكتورة

هاد الكلاس **GanttCanvas** بيرسم الـ Gantt **وهو بيتحرك**. الدالة **play** بتقسم الوقت الكلي على إطارات، 40 إطار بالثانية، وكل 25 جزء من الألف من الثانية بتزيد المتغيّر **now** وبترجع ترسم. ودالة الرسم بترسم بس الجزء اللي قبل **now**، وبتحط خط أحمر عند الوقت الحالي، وبتكتب فوق مين على المعالج هسا. استخدمت **after** بدل **sleep** عشان الشاشة ما تعلّق وهي بتتحرك. وكل process إلها لون ثابت، والـ idle رمادي فاتح، والـ context switch مخطّط.

زر Run: الواجهة ما بتحسب اشي لحالها

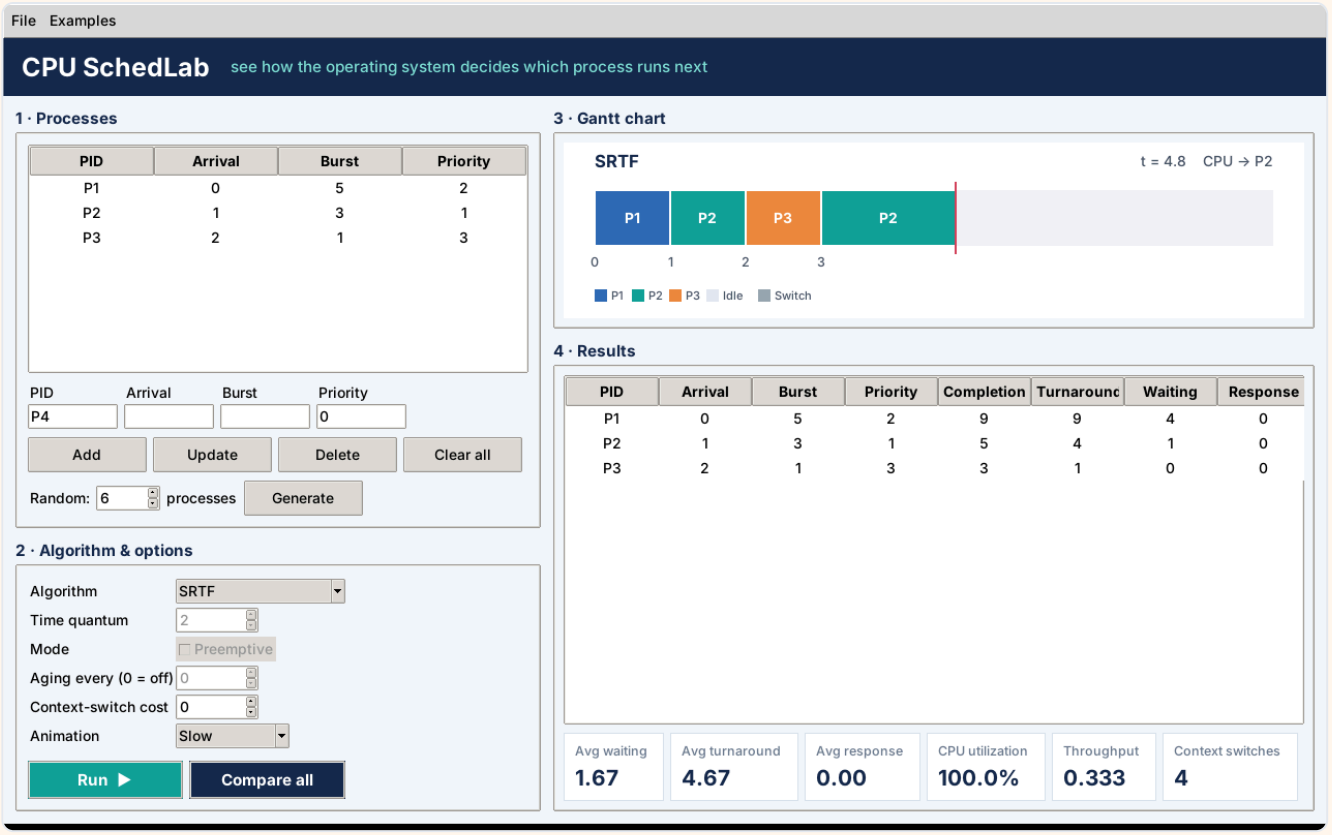
CPU_SchedLab.py lines 725–742

```python
    def on_run(self):
        """The Run button: core -> metrics -> draw. The GUI calculates nothing itself."""
        try:
            result = run(self.algorithm.get(), self.processes, self.read_options())
        except ValueError as err:
            return messagebox.showerror("Cannot run", str(err))
        rows, summary = compute_metrics(result)
        self.last = (result, rows, summary)
        self.gantt.play(result, SPEEDS[self.speed.get()])
        print(result.algorithm, "\n" + text_gantt(result.timeline), "\n")   # simple Gantt
        self.rtable.delete(*self.rtable.get_children())
        for r in rows:
            self.rtable.insert("", "end", values=list(r.values()))
        for key, label in self.cards.items():
            v = summary[key]
            label.config(text=f"{v:.1f}%" if key == "cpu_utilization" else
                         f"{v:.3f}" if key == "throughput" else
                         f"{v:.2f}" if isinstance(v, float) else str(v))
```

🎤 شو بتحكي للدكتورة

هاي الدالة بتشتغل لما نكبس زر **Run**، وبتوضّح إنه الواجهة **ما بتحسب اشي لحالها**. بتقرأ شو اختار المستخدم، وبعدين بتنادي **run** من الـ core عشان تشغّل الخوارزمية، وبعدها **compute_metrics** عشان تحسب المقاييس. وبعدين بترسم الـ Gantt المتحرك، وبتعبّي جدول النتائج والمربعات تبعت المعدّلات. وإذا في غلط بالإدخال، زي burst صفر أو حرف بدل رقم، بتطلع رسالة خطأ واضحة بدل ما يوقف البرنامج.

🖥️ شو بتعرض عالشاشة



بورّي الشاشة وأقسامها الأربعة: (1) جدول الـ processes مع Add و Update و Delete و (2) Generate، الخوارزمية وخياراتها، وهي بتتفعّل حسب الخوارزمية، (3) الـ Gantt المتحرك، (4) النتائج. بكبس Run على سرعة Slow عشان تبيّن الحركة والخط الأحمر، وبعدين بكبس عالرسمة عشان تكمل مرة وحدة.

حفظ وتحميل وتصدير CSV (F12)

CPU_SchedLab.py lines 466–491

```python
def save_workload(path, processes):
    """F12 Save the processes in a CSV file (opens in Excel)."""
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["pid", "arrival", "burst", "priority"])
        for p in processes:
            writer.writerow([p.pid, p.arrival, p.burst, p.priority])


def load_workload(path):
    """F12 Read processes back from a CSV file."""
    with open(path, newline="", encoding="utf-8") as f:
        return [make_process(r["pid"], r["arrival"], r["burst"], r.get("priority") or "0")
                for r in csv.DictReader(f)]


def export_results(path, result, rows, summary):
    """F12 Export the Gantt timeline, the results table and the averages as CSV."""
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["Algorithm", result.algorithm])
        w.writerow(["pid", "start", "end"])
        w.writerows([s.pid, s.start, s.end] for s in result.timeline)
        w.writerow(list(rows[0].keys()))
        w.writerows(list(r.values()) for r in rows)
        w.writerows([k, round(v, 3) if isinstance(v, float) else v] for k, v in summary.items())
```

🎤 شو بتحكي للدكتورة

هاي وظيفة **F12**، وبتمثّل عمليات **نظام الملفات**. الدالة **save_workload** بتحفظ الـ processes بملف **CSV**، وهو جدول نصي بينفتح على Excel. و**load_workload** بتقرأ الملف وبترجّع الـ processes، وكل قيمة بتمرق على make_process عشان نتأكد إنه الملف سليم. أما **export_results** فبتصدّر الخط الزمني وجدول النتائج والمعدّلات بملف واحد. وكل هاد موجود بقائمة **File** بالواجهة.